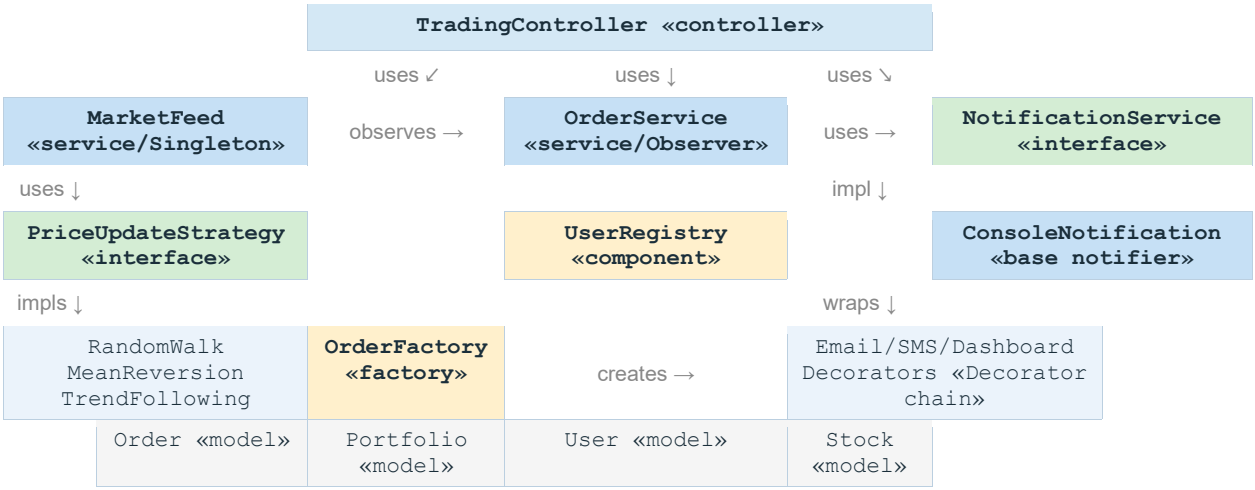
TradeSimulator v2.0

Design Document — Week 2

Item	URL
Backend Repo	https://github.com/isutariy-P532-SPRING2026/TradeSimulator_V2_backend
Frontend Repo	https://github.com/isutariy-P532-SPRING2026/TradeSimulator_V2_frontend
Live App	https://isutariy-p532-spring2026.github.io/TradeSimulator_V2_frontend/
Backend API	https://tradesimulator-v2-backend.onrender.com

1. Component Diagram

The diagram below shows all main classes, their packages, and their relationships. Boxes are classes/interfaces; labels on arrows indicate the nature of the dependency.



New classes added in Week 2 are shaded in light yellow (UserRegistry) or light blue (new strategies and decorators). The core Week 1 classes (MarketFeed, OrderService, TradingController) are unchanged structurally.

2. Design Pattern Justifications

Pattern	Classes Involved	Change It Protects Against
Strategy	PriceUpdateStrategy (interface) RandomWalkStrategy MeanReversionStrategy TrendFollowingStrategy injected into MarketFeed	Adding a new pricing algorithm requires only a new class implementing PriceUpdateStrategy; MarketFeed never changes.
Observer	PriceObserver (interface) OrderService implements PriceObserver registered via MarketFeed.addObserver()	Adding new price-change reactions (risk monitor, analytics) requires only a new PriceObserver; MarketFeed never changes.
Decorator	NotificationDecorator (abstract) ConsoleNotificationService (base) EmailNotificationDecorator SmsNotificationDecorator BadgeDashboardDecorator MutableNotificationService	Adding notification channels requires only a new decorator class; existing notifiers and OrderService are never modified.
Factory	OrderFactory.createOrder() returns Order with correct OrderType and OrderSide	Adding new order types or stricter validation is confined to OrderFactory; all callers receive a correctly built Order.
Singleton	MarketFeed (@Service) Spring manages exactly one instance. UserRegistry (@Component) One registry owns all per-user state.	Multiple instances would track divergent prices; singleton guarantees portfolio valuations and observer lists remain consistent.

3. Week 2 Changes — Pre-existing Files Modified

For each deliverable change, the table below lists every pre-existing source file that required modification. New files added to implement a feature are not counted.

Change	Pre-existing Files Modified	Count
Change 1 Pricing Algorithms	MarketFeed.java — removed final keyword from pricingStrategy field; added setStrategy() method	**1**
Change 2 Multi-channel Notifications	NotificationConfig.java — emptied bean definition to prevent Spring conflict with UserRegistry-managed notification chains	**1**
Change 3 Multiple Users	TradingController.java — replaced Portfolio/OrderService injection with UserRegistry; added userId query param to all endpoints. OrderService.java — removed @Service annotation so UserRegistry can create one instance per user.	**2**
CI/CD Pipeline	No changes required — existing ci.yml already satisfies all Week 2 requirements.	**0**
TOTAL		**4**

Target: 0–1 modified files per change. Actual average: 1.0 modified file per change.

4. New Files Added (Week 2)

All files below are brand-new additions. They do not count against the design-quality score.

New File	Change	Purpose
MeanReversionStrategy.java	Change 1	Implements PriceUpdateStrategy — price drifts toward moving average
TrendFollowingStrategy.java	Change 1	Implements PriceUpdateStrategy — price continues recent momentum
StrategyController.java	Change 1	REST endpoint to list and switch pricing strategies at runtime
EmailNotificationDecorator.java	Change 2	Decorator — logs a mock email notification
SmsNotificationDecorator.java	Change 2	Decorator — logs a mock SMS notification
BadgeDashboardDecorator.java	Change 2	Decorator — increments per-user badge counter in DashboardAlertStore
NotificationChannels.java	Change 2	Enum — CONSOLE, EMAIL, SMS, DASHBOARD
NotificationBuilder.java	Change 2	Builds the decorator chain from a user's selected channels
MutableNotificationService.java	Change 2	Wrapper that allows swapping the notification chain at runtime
DashboardAlertStore.java	Change 2	Holds per-user unread alert counts; injected into controller
User.java	Change 3	Model — holds userId, name, and selected notification channels
UserRegistry.java	Change 3	Component — owns one Portfolio, OrderService, and MutableNotificationService per user
CorsConfig.java	Infrastructure	Allows cross-origin requests from GitHub Pages to Render

6. Pattern Audit

Strategy

PriceUpdateStrategy is an interface injected into MarketFeed; RandomWalkStrategy, MeanReversionStrategy, and TrendFollowingStrategy each implement it. Adding Change 1 required only creating two new classes and exposing a setStrategy() setter on MarketFeed — the scheduling loop and observer notifications were untouched.

Observer

PriceObserver is an interface with a single onPriceUpdate(Stock) method; OrderService registers itself with MarketFeed.addObserver() at construction time. This decoupling meant that per-user OrderService instances (Change 3) registered themselves independently without any modification to MarketFeed.

Decorator

NotificationDecorator is an abstract class wrapping NotificationService; ConsoleNotificationService is the base, and Email, SMS, and Dashboard decorators each extend it and call super first. NotificationBuilder assembles the chain from a user's selected channels, and MutableNotificationService allows the chain to be swapped at runtime without touching OrderService.

Factory

OrderFactory.createOrder() is the single point where ticker, quantity, side, orderType, and limitPrice are validated and assembled into an Order. All callers — including TradingController — delegate to this method, so any future order-type additions or input validation changes are confined to one class.

Singleton

MarketFeed is annotated @Service so Spring creates and injects exactly one instance; all per-user OrderService instances share this one feed, ensuring price consistency. UserRegistry is a @Component singleton that owns the canonical map of users to their Portfolio, OrderService, and notification chain.